Java Collections Framework – Practice Problem Statements

The practice sheet will cover:

1. Set Interface

- HashSet
- LinkedHashSet
- TreeSet

2. Map Interface

- HashMap
- LinkedHashMap
- TreeMap

Module 11: Java Problem Statements Using Set and Map Interfaces

Instructions for Students

- Solve each problem using the specified collection class.
- Use the methods of the respective classes wherever applicable.
- Do not use arrays or other collection classes as substitutes for the specified collection.
- Display the collection after every major modification.
- Handle duplicate elements and missing keys appropriately.
- Wherever possible, implement the same problem using different implementations and compare their behaviour.

Part A – Set Interface

1. Student Registration Management System (HashSet)

Develop a Java application to manage student registration numbers using `HashSet`.

Requirements:

1. Create a `HashSet<Integer>` to store student registration numbers.
2. Add at least 10 registration numbers using `add()`.
3. Attempt to insert duplicate registration numbers and observe the result.
4. Create another `HashSet` containing registration numbers of newly admitted students.
5. Insert all new registration numbers using `addAll()`.
6. Check whether a particular registration number exists using `contains()`.
7. Check whether all registration numbers from another set exist using `containsAll()`.
8. Remove a particular registration number using `remove()`.
9. Remove multiple registration numbers using `removeAll()`.
10. Retain only the registration numbers present in another set using `retainAll()`.
11. Display the total number of registered students using `size()`.
12. Check whether the set is empty using `isEmpty()`.
13. Traverse the set using an enhanced for loop and an `Iterator`.
14. Convert the set into an array using `toArray()`.
15. Remove all registration records using `clear()`.

2. Unique Product Inventory System (LinkedHashSet)

Develop an application to maintain a list of unique products in an inventory using `LinkedHashSet<String>`.

Requirements:

1. Add 10 product names using `add()`.
2. Insert duplicate product names and observe how duplicates are handled.
3. Maintain the insertion order while displaying products.
4. Add another collection of products using `addAll()`.
5. Search for a product using `contains()`.
6. Remove a discontinued product using `remove()`.
7. Remove multiple discontinued products using `removeAll()`.
8. Retain only products available in a particular warehouse using `retainAll()`.
9. Display the number of unique products using `size()`.
10. Traverse products using an `Iterator`.
11. Traverse products using `forEach()`.
12. Convert the collection into an array using `toArray()`.
13. Create a copy of the existing collection using the `LinkedHashSet` constructor.
14. Clear the inventory using `clear()`.

3. Employee Salary Ranking System (TreeSet)

Develop an application to store and manage unique employee salaries using `TreeSet<Integer>`.

Requirements:

1. Insert at least 10 different employee salaries.
2. Insert duplicate salary values and observe the result.
3. Display salaries in ascending order.
4. Display salaries in descending order using `descendingSet()`.
5. Find the lowest and highest salaries using `first()` and `last()`.
6. Find the salary immediately lower than a given salary using `lower()`.
7. Find the salary lower than or equal to a given salary using `floor()`.
8. Find the salary higher than a given salary using `higher()`.
9. Find the salary higher than or equal to a given salary using `ceiling()`.
10. Remove and retrieve the lowest salary using `pollFirst()`.
11. Remove and retrieve the highest salary using `pollLast()`.
12. Display salaries within a specified range using `subSet()`.
13. Display salaries below a specified value using `headSet()`.
14. Display salaries above a specified value using `tailSet()`.
15. Retrieve elements using `descendingIterator()`.
16. Identify the comparator used for sorting through `comparator()`.

4. Online Examination Subject Selection (HashSet vs LinkedHashSet vs TreeSet)

Develop an application that allows students to select subjects for an online examination.

Requirements:

1. Create three collections: `HashSet`, `LinkedHashSet`, and `TreeSet`.
2. Add the same set of subject names to all three collections.
3. Insert duplicate subject names.
4. Display the contents of all three collections.
5. Compare the order in which elements are displayed.
6. Search for a particular subject using `contains()`.
7. Remove a subject using `remove()`.
8. Display the size of each collection.
9. Demonstrate union, intersection, and difference using `addAll()`, `retainAll()`, and `removeAll()`.
10. Explain the differences in duplicate handling, insertion order, and sorting behaviour.

5. Library Book Category Management (TreeSet)

Develop a library application to maintain unique book categories in sorted order.

Requirements:

1. Insert categories such as Java, Python, C++, Database, Networking, and AI.
2. Display all categories in alphabetical order.
3. Add new categories and avoid duplicates.
4. Find the first and last categories.
5. Find categories alphabetically lower or higher than a given category using `lower()`, `higher()`, `floor()`, and `ceiling()`.
6. Display categories within a specified alphabetical range using `subSet()`.
7. Display categories before and after a given category using `headSet()` and `tailSet()`.
8. Display categories in reverse order.
9. Remove the first and last categories using `pollFirst()` and `pollLast()`.
10. Create a `TreeSet` using a custom `Comparator` to display categories in descending alphabetical order.

6. Course Enrollment Comparison (Set Operations)

A university offers different courses to students enrolled in two different sections.

Develop an application using `HashSet<Integer>` to identify common and distinct student enrollments.

Requirements:

1. Create two sets representing students enrolled in Section A and Section B.
2. Insert student registration numbers into both sets.
3. Identify students enrolled in either section using `addAll()`.
4. Identify students enrolled in both sections using `retainAll()`.
5. Identify students enrolled only in Section A using `removeAll()`.
6. Identify students enrolled only in Section B.
7. Check whether one section's enrollment is completely contained in another using `containsAll()`.
8. Check the total number of unique students.
9. Display all results using appropriate traversal techniques.

Part B – Map Interface

7. Student Marks Management System (HashMap)

Develop a Java application to manage student marks using `HashMap<Integer, Integer>`, where the key represents the student roll number and the value represents marks.

Requirements:

1. Insert at least 10 student records using `put()`.
2. Insert a duplicate roll number with a different marks value and observe the result.
3. Add multiple student records using `putAll()`.
4. Retrieve marks using `get()`.
5. Retrieve marks for a non-existing roll number using `getOrDefault()`.
6. Check whether a particular roll number exists using `containsKey()`.
7. Check whether a particular marks value exists using `containsValue()`.
8. Update marks using `put()`.
9. Update marks using `replace()`.
10. Replace marks only when the existing marks match a specified value using `replace(key, oldValue, newValue)`.
11. Remove a student record using `remove()`.
12. Remove a student record only if its key-value pair matches using `remove(key, value)`.
13. Display all roll numbers using `keySet()`.
14. Display all marks using `values()`.
15. Display all student records using `entrySet()`.
16. Traverse records using `forEach()`.
17. Display the total number of students using `size()`.
18. Check whether the map is empty using `isEmpty()`.
19. Remove all records using `clear()`.

8. Online Shopping Cart Management (HashMap)

Develop an online shopping cart application using `HashMap<String, Integer>`, where the key represents the product name and the value represents its quantity.

Requirements:

1. Add products and their quantities using `put()`.
2. Add multiple products using `putAll()`.
3. Update the quantity of an existing product.
4. Retrieve the quantity using `get()`.
5. Retrieve the quantity of a product that does not exist using `getOrDefault()`.
6. Check whether a product exists using `containsKey()`.
7. Check whether a particular quantity exists using `containsValue()`.
8. Increase the quantity of an existing product using `merge()`.

9. Add a product only if it does not exist using `putIfAbsent()`.
10. Update product quantities using `compute()`.
11. Update the quantity only if the product exists using `computeIfPresent()`.
12. Add a default quantity if a product does not exist using `computeIfAbsent()`.
13. Remove a product using `remove()`.
14. Display all products using `keySet()`.
15. Display all quantities using `values()`.
16. Display product names with quantities using `entrySet()`.
17. Calculate the total number of items in the cart using the values.
18. Display the number of unique products using `size()`.

9. Employee Department Management System (LinkedHashMap)

Develop an application to maintain employee records in their insertion order using `LinkedHashMap<Integer, String>`.

Requirements:

1. Insert employee ID and employee name using `put()`.
2. Add multiple employee records using `putAll()`.
3. Display employee records in insertion order.
4. Update an employee name using `replace()`.
5. Retrieve employee details using `get()` and `getOrDefault()`.
6. Check whether an employee ID exists using `containsKey()`.
7. Check whether an employee name exists using `containsValue()`.
8. Remove an employee record using `remove()`.
9. Display all employee IDs using `keySet()`.
10. Display all employee names using `values()`.
11. Traverse employee records using `entrySet()` and `forEach()`.
12. Demonstrate the difference between `HashMap` and `LinkedHashMap` by inserting identical records into both.
13. Create another `LinkedHashMap` with access-order enabled using the appropriate constructor.
14. Access different employee records using `get()` and observe how the order changes.
15. Override `removeEldestEntry()` in a custom `LinkedHashMap` class to maintain only the latest five employee records.

10. Product Price Management System (TreeMap)

Develop an application to maintain product prices using `TreeMap<Integer, Double>`, where the key represents the product ID and the value represents its price.

Requirements:

1. Insert at least 10 product records using `put()`.

2. Add multiple records using `putAll()`.
3. Display products in ascending order of product IDs.
4. Display products in descending order using `descendingMap()`.
5. Retrieve the first and last product IDs using `firstKey()` and `lastKey()`.
6. Retrieve the first and last key-value pairs using `firstEntry()` and `lastEntry()`.
7. Find the product ID immediately lower than a given ID using `lowerKey()`.
8. Find the product ID lower than or equal to a given ID using `floorKey()`.
9. Find the product ID higher than a given ID using `higherKey()`.
10. Find the product ID higher than or equal to a given ID using `ceilingKey()`.
11. Retrieve entries using `lowerEntry()`, `floorEntry()`, `higherEntry()`, and `ceilingEntry()`.
12. Remove and retrieve the first entry using `pollFirstEntry()`.
13. Remove and retrieve the last entry using `pollLastEntry()`.
14. Display products within a specified ID range using `subMap()`.
15. Display products below a specified ID using `headMap()`.
16. Display products above a specified ID using `tailMap()`.
17. Retrieve keys using `navigableKeySet()` and `descendingKeySet()`.
18. Display the comparator used by the map using `comparator()`.

11. Word Frequency Counter (HashMap)

Develop a Java application to count the frequency of each word in a given paragraph using `HashMap<String, Integer>`.

Requirements:

1. Accept a paragraph from the user.
2. Split the paragraph into individual words.
3. Store each unique word as a key and its frequency as a value.
4. Insert words using `put()`.
5. Use `getOrDefault()` to calculate word frequencies.
6. Use `merge()` to update the frequency of each word.
7. Use `compute()` to update word frequencies.
8. Use `computeIfAbsent()` and `computeIfPresent()` to demonstrate their functionality.
9. Display all unique words using `keySet()`.
10. Display all frequencies using `values()`.
11. Display each word with its frequency using `entrySet()`.
12. Identify the word having the highest frequency.
13. Remove a particular word using `remove()`.
14. Display the total number of unique words using `size()`.

12. University Student Ranking System (TreeMap)

Develop an application to maintain student marks in sorted order using `TreeMap<Integer, String>`, where the key represents marks and the value represents the student name.

Requirements:

1. Insert student marks and names using `put()`.
2. Display students in ascending order of marks.
3. Display students in descending order using `descendingMap()`.
4. Retrieve the lowest and highest marks using `firstKey()` and `lastKey()`.
5. Retrieve the corresponding student records using `firstEntry()` and `lastEntry()`.
6. Find the nearest lower and higher marks using `lowerEntry()` and `higherEntry()`.
7. Find the nearest lower or equal and higher or equal marks using `floorEntry()` and `ceilingEntry()`.
8. Display student records within a specified marks range using `subMap()`.
9. Display students scoring below a specified mark using `headMap()`.
10. Display students scoring above a specified mark using `tailMap()`.
11. Remove the student with the lowest marks using `pollFirstEntry()`.
12. Remove the student with the highest marks using `pollLastEntry()`.
13. Traverse records using `entrySet()` and `forEach()`.
14. Create a custom `TreeMap` using a `Comparator` to arrange marks in descending order.

Note: Since a Map cannot contain duplicate keys, students having identical marks will overwrite each other in this particular representation. Consider an alternative design if all students with the same marks must be retained.

Part C – Comparative and Advanced Programming Exercises

13. Employee Record Management (Comparison of Map Implementations)

Develop an application using `HashMap`, `LinkedHashMap`, and `TreeMap` to store employee IDs and employee names.

Requirements:

1. Insert identical employee records into all three maps.
2. Insert duplicate employee IDs with updated employee names.
3. Display records using `entrySet()`.
4. Compare the iteration order of all three maps.
5. Search for an employee using `containsKey()` and `get()`.
6. Remove an employee using `remove()`.
7. Update employee details using `replace()`.
8. Display all keys and values.
9. Demonstrate the use of `forEach()`.
10. Compare the three implementations based on ordering, performance characteristics, and supported navigation operations.

14. Student Attendance Tracking System (HashMap and LinkedHashMap)

Develop an attendance tracking application using student roll numbers as keys and attendance status as values.

Requirements:

1. Insert student attendance records using `put()`.
2. Update attendance status using `replace()`.
3. Use `putIfAbsent()` to insert a student only if the roll number does not exist.
4. Use `computeIfPresent()` to update an existing attendance record.
5. Use `computeIfAbsent()` to initialize missing attendance records.
6. Use `merge()` to maintain attendance counts across multiple sessions.
7. Use `replaceAll()` to update all attendance statuses based on a given condition.
8. Use `getOrDefault()` to handle students whose attendance records are unavailable.
9. Display students using `keySet()`, `values()`, and `entrySet()`.
10. Compare the output using `HashMap` and `LinkedHashMap`.

Part D – Method Coverage Checklist

The following checklist can be used to ensure that students have revised the important methods of each implementation.

Collection	Important methods to practise
HashSet	`add()`, `addAll()`, `remove()`, `removeAll()`, `retainAll()`, `contains()`, `containsAll()`, `size()`, `isEmpty()`, `clear()`, `iterator()`, `forEach()`, `toArray()`
LinkedHashSet	All common Set methods, insertion-order traversal, constructors, `spliterator()`, `stream()`
TreeSet	All common Set methods, `first()`, `last()`, `lower()`, `floor()`, `higher()`, `ceiling()`, `pollFirst()`, `pollLast()`, `descendingSet()`, `descendingIterator()`, `subSet()`, `headSet()`, `tailSet()`, `comparator()`
HashMap	`put()`, `putAll()`, `get()`, `getOrDefault()`, `putIfAbsent()`, `remove()`, `replace()`, `replaceAll()`, `containsKey()`, `containsValue()`, `keySet()`, `values()`, `entrySet()`, `compute()`, `computeIfAbsent()`, `computeIfPresent()`, `merge()`, `forEach()`
LinkedHashMap	All common Map methods, insertion order, access order, `removeEldestEntry()`
TreeMap	All common Map methods, `firstKey()`, `lastKey()`, `firstEntry()`, `lastEntry()`, `lowerKey()`, `floorKey()`, `higherKey()`, `ceilingKey()`, `lowerEntry()`, `floorEntry()`, `higherEntry()`, `ceilingEntry()`, `pollFirstEntry()`, `pollLastEntry()`, `descendingMap()`, `navigableKeySet()`, `descendingKeySet()`, `subMap()`, `headMap()`, `tailMap()`, `comparator()`

Important points for students:

- `HashSet` and `HashMap` do not guarantee iteration order.
- `LinkedHashSet` and `LinkedHashMap` maintain insertion order by default.
- `TreeSet` and `TreeMap` maintain sorted order.
- Set implementations do not allow duplicate elements.
- Map implementations do not allow duplicate keys, but multiple keys can have the same value.
- `TreeSet` and `TreeMap` support navigation and range-based operations.
- Students should also practise using custom `Comparator` implementations.

Note: The practice sheet uses standard Java Collections Framework APIs. Some methods are available through inherited interfaces, while others are specific to NavigableSet, NavigableMap, or LinkedHashMap. The exercises assume Java 8 or later.